MVP争夺战

题目描述

在星球争霸篮球赛对抗赛中，最大的宇宙战队希望每个人都能拿到 MVP ，MVP 的条件是单场最高分得分获得者。

可以并列所以宇宙战队决定在比赛中尽可能让更多队员上场，并且让所有得分的选手得分都相同，然而比赛过程中的每一分钟的得分都只能由某一个人包揽。

输入描述

输入第一行为一个数字 t，表示为有得分的分钟数

- 1 ≤ t ≤ 50

第二行为 t 个数字，代表每一分钟的得分 p

- 1 ≤ p ≤ 50

输出描述

输出有得分的队员都是 MVP 时，最少得 MVP 得分。

用例

输入	9 5 2 1 5 2 1 5 2 1
输出	6
说明	样例解释 一共 4 人得分，分别都是 6 分 5 + 1 ， 5 + 1 ， 5 + 1 ， 2 + 2 + 2

题目解析

本题和

LeetCode - 698 划分为k个相等的子集_伏城之外的博客-CSDN博客

属于同一类问题，解法相同，题解请看上面博客：划分k个相等的子集

JavaScript　算法源码

运行

```
const rl = require("readline").createInterface({ input: process.stdin });
var iter = rl[Symbol.asyncIterator]();
const readline = async () => (await iter.next()).value;

void (async function () {
  const n = parseInt(await readline());
  const arr = (await readline()).split(" ").map(Number);

  console.log(solution(arr, n));
})();

function solution(arr, n) {
  const sum = arr.sort((a, b) => b - a).reduce((p, c) => p + c);

  let count = n;
  while (count >= 1) {
    // 由于canPartition方法中会删除arr元素，因此我们不能直接传递arr过去，需要传递arr备份，否则会影响下一次count判断
    if (canPartition([...arr], sum, count)) {
      return sum / count;
    } else {
      count--;
    }
  }
}

function canPartition(arr, sum, count) {
  if (sum % count) return false;
```

```
28
29    let subSum = sum / count;
30
31    if (subSum < arr[0]) return false;
32
33    while (arr[0] === subSum) {
34      arr.shift();
35      count--;
36    }
37
38    const buckets = new Array(count).fill(0);
39
40    return partition(0, arr, subSum, buckets);
41  }
42
43  function partition(index, arr, subSum, buckets) {
44    if (index === arr.length) {
45      return true;
46    }
47
48    const select = arr[index];
49
50    for (let i = 0; i < buckets.length; i++) {
51      if (i > 0 && buckets[i] === buckets[i - 1]) continue;
52      if (buckets[i] + select <= subSum) {
53        buckets[i] += select;
54        if (partition(index + 1, arr, subSum, buckets)) return true;
55        buckets[i] -= select;
56      }
57    }
58
59    return false;
60  }
```

Java算法源码

```java
import java.util.LinkedList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = sc.nextInt();

        LinkedList<Integer> link = new LinkedList<>();
        for (int i = 0; i < m; i++) {
            link.add(sc.nextInt());
        }

        System.out.println(solution(link, m));
    }

    public static int solution(LinkedList<Integer> link, int m) {
        link.sort((a, b) -> b - a);

        int sum = 0;
        for (Integer ele : link) {
            sum += ele;
        }

        while (m >= 1) {
            // 由于canPartition方法中会删除link元素，因此我们不能直接传递link过去，需要传递link备份，否则会影响下一次count判断
            LinkedList<Integer> link_cp = new LinkedList<>(link);
            if (canPartitionMSubsets(link_cp, sum, m)) return sum / m;
            m--;
        }

        return sum;
    }

```

```java
    public static boolean canPartitionMSubsets(LinkedList<Integer> link, int sum, int m) {
        if (sum % m != 0) return false;
        int subSum = sum / m;

        if (subSum < link.get(0)) return false;

        //    while (link.get(0) == subSum) { // 此段代码可能会出现越界
        while (!link.isEmpty() && link.get(0) == subSum) {
            link.removeFirst();
            m--;
        }

        int[] buckets = new int[m];
        return partition(link, 0, buckets, subSum);
    }

    public static boolean partition(LinkedList<Integer> link, int index, int[] buckets, int subSum) {
        if (index == link.size()) return true;

        int select = link.get(index);

        for (int i = 0; i < buckets.length; i++) {
            if (i > 0 && buckets[i] == buckets[i - 1]) continue;
            if (select + buckets[i] <= subSum) {
                buckets[i] += select;
                if (partition(link, index + 1, buckets, subSum)) return true;
                buckets[i] -= select;
            }
        }

        return false;
    }
}
```

Python算法源码

```python
# 输入获取
m = int(input())
link = list(map(int, input().split()))


# 算法入口
def getResult(link, m):
    link.sort(reverse=True)

    sumV = 0
    for ele in link:
        sumV += ele

    while m >= 1:
        if canPartitionMSubsets(link[:], sumV, m):
            return int(sumV / m)
        m -= 1

    return sumV


def canPartitionMSubsets(link, sumV, m):
    if sumV % m != 0:
        return False

    subSum = sumV / m

    if subSum < link[0]:
        return False

    while len(link) > 0 and link[0] == subSum:
        link.pop(0)
        m -= 1

    buckets = [0] * m
```

```
    return partition(link, 0, buckets, subSum)


def partition(link, index, buckets, subSum):
    if index == len(link):
        return True

    select = link[index]

    for i in range(len(buckets)):
        if i > 0 and buckets[i] == buckets[i - 1]:
            continue

        if select + buckets[i] <= subSum:
            buckets[i] += select
            if partition(link, index + 1, buckets, subSum):
                return True
            buckets[i] -= select

    return False


# 算法调用
print(getResult(link, m))
```

C算法源码

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX_M 51

bool partition(const int link[], int link_size, int index, int buckets[], int buckets_size, int subSum) {
```

```
    if (index == link_size) {
        return true;
    }

    int select = link[index];

    for (int i = 0; i < buckets_size; i++) {
        if (i > 0 && buckets[i] == buckets[i - 1]) continue;

        if (select + buckets[i] <= subSum) {
            buckets[i] += select;
            if (partition(link, link_size, index + 1, buckets, buckets_size, subSum)) {
                return true;
            }
            buckets[i] -= select;
        }
    }

    return false;
}

bool canPartitionMSubsets(int link[], int link_size, int sum, int buckets_size) {
    if (sum % buckets_size != 0) return false;

    int subSum = sum / buckets_size;
    if (subSum < link[0]) return false;

    int buckets[MAX_M] = {0};
    return partition(link, link_size, 0, buckets, buckets_size, subSum);
}

int cmp(const void *a, const void *b) {
    return *((int *) b) - *((int *) a);
}

int solution(int link[], int link_size) {
    int sum = 0;
```

```
    for (int i = 0; i < link_size; i++) {
        sum += link[i];
    }

    qsort(link, link_size, sizeof(link[0]), cmp);

    int buckets_size = link_size;
    while (buckets_size >= 1) {
        int link_cp[MAX_M];
        for (int i = 0; i < link_size; i++) link_cp[i] = link[i];

        if (canPartitionMSubsets(link_cp, link_size, sum, buckets_size)) {
            return sum / buckets_size;
        }

        buckets_size--;
    }
}

int main() {
    int m;
    scanf("%d", &m);

    int link[MAX_M] = {0};
    for (int i = 0; i < m; i++) {
        scanf("%d", &link[i]);
    }

    printf("%d\n", solution(link, m));

    return 0;
}
```

C++算法源码

```cpp
#include <bits/stdc++.h>

using namespace std;

bool partition(deque<int> &link, int index, vector<int> &buckets, int subSum) {
    if (index == link.size()) return true;

    int select = link[index];

    for (int i = 0; i < buckets.size(); i++) {
        if (i > 0 && buckets[i] == buckets[i - 1]) continue;

        if (select + buckets[i] <= subSum) {
            buckets[i] += select;
            if (partition(link, index + 1, buckets, subSum)) return true;
            buckets[i] -= select;
        }
    }

    return false;
}

bool canPartitionMSubsets(deque<int> &link, int sum, int m) {
    if (sum % m != 0) return false;

    int subSum = sum / m;

    if (subSum < link[0]) return false;

    while (!link.empty() && link[0] == subSum) {
        link.pop_front();
        m--;
    }

    vector<int> buckets(m, 0);
    return partition(link, 0, buckets, subSum);
}

```

```cpp
int solution(deque<int> &link, int m) {
    sort(link.begin(), link.end());
    reverse(link.begin(), link.end());

    int sum = 0;
    for (const auto &item: link) {
        sum += item;
    }

    while (m >= 1) {
        deque<int> link_cp = link;
        if (canPartitionMSubsets(link_cp, sum, m)) return sum / m;
        m--;
    }

    return sum;
}

int main() {
    int m;
    cin >> m;

    deque<int> link(m);
    for (int i = 0; i < m; i++) {
        cin >> link[i];
    }

    cout << solution(link, m) << endl;

    return 0;
}
```